

N-Puzzle 问题求解研究报告

2026 年秋季学期

姓名：林子航　　学号：24070031012

姓名：李行　　学号：24070031008
姓名：张文璟　　学号：24020007159
姓名：周以撒　　学号：24040021050

目　录

1. 启发式算法综述.....1
1.1 算法描述.....1
1.2 算法的可采纳性.....2
1.2.1 A* 算法.....2
1.2.2 A* 算法可采纳性证明.....2
1.3 重复扩展结点的问题.....3
1.3.1 重复扩展结点问题.....3
1.3.2 启发函数的单调性.....3
1.4 IDA* 算法基本原理及其可采纳性.....3
2. N-Puzzle 游戏.....4
2.1 N-Puzzle 游戏描述.....4
2.2 游戏的建模、状态及状态空间、状态转换规则.....5
2.3 N-Puzzle 可解性研究.....5
2.4 三个启发函数的有效性和单调性证明.....6
2.4.1 曼哈顿距离.....6
2.4.2 不在位将牌（错位将牌数）.....6
2.4.3 Disjoint Pattern（不相交模式数据库）.....6
3. N-Puzzle 求解算法实现.....8
3.1 状态及结点的数据结构.....8
状态：PuzzleBoard，用 64 位整数把盘面打包起来.....8
结点：Node（框架提供）.....8
3.2 Open 表及 Closed 表的数据结构.....9
Closed 表：框架的 explored.....9
Open 表：PqFrontier，二叉堆加索引，再配惰性删除和定期压实.....9
3.3 三个启发函数的具体实现.....10
3.3.1 曼哈顿距离.....10
3.3.2 不在位将牌.....10
3.3.3 Disjoint Pattern 和 Pattern Database.....10
3.4 求解后继结点的函数.....11
4. N-Puzzle 问题求解探讨（实验数据分析）.....12
4.1 Open 表及 Closed 表数据结构的改进及其对算法性能的影响.....12
4.2 A *算法*、*IDA* 算法的性能比较.....13
4.2.1 与参考程序的逐项对标.....13
4.2.2 A *与 IDA* 的对比分析.....14
4.3 三个不同的启发函数及 IDA* 的性能比较.....14
4.3.1 运行时间对比及分析.....14
4.3.2 扩展结点比较及分析.....15
4.3.3 模式组并非越大越强.....16
4.3.4 阶段三倒序实例的求解.....16
5. 解路径的可视化.....17
5.1 设计目标与总体结构.....17
5.2 绘制层：BoardPanel 的关键逻辑.....18

5.3 控制层：PuzzleVisualizer 的关键逻辑.....18
5.4 演示效果.....19
6. 总结及展望.....20
6.1 完成情况对照.....20
6.2 主要收获.....20
6.3 不足与展望.....20
7. 参考文献.....21

1. 启发式算法综述

1.1 算法描述

状态空间搜索解决问题的思路很直接：从初始状态出发，不断套用动作算子生成后继状态，直到撞上目标状态。整个过程中要维护两张表：

- Open 表（本框架里叫 Frontier，待扩展集合），存放已经生成、但还没扩展的结点；
- Closed 表（本框架里叫 explored，已扩展集合），存放已经扩展过的状态。

不同搜索算法之间的差别其实只有一条：**从 Open 表里挑哪个结点先扩展**。本框架用一个估价函数把这条规则统一描述出来：

$$f(n) = g(n) + h(n)$$

其中 $g(n)$ 是从初始状态走到结点 n 实际花了多少代价，$h(n)$ 是从 n 到目标还差多少的估计值。三种经典算法都是它的特例（框架里用 `EvaluationType` 枚举表示）：

算法	取值方式	框架枚举	特点
Dijkstra（一致代价搜索）	$f = g$，即 $h \equiv 0$	`PATH_COST`	保证最优，但太盲目，扩展的结点数非常多
贪心最佳优先搜索	$f = h$，即 $g \equiv 0$	`HEURISTIC`	跑得快，但不保证最优
A*	$f = g + h$	`FULL`	h 可采纳时保证最优，效率比 Dijkstra 高得多

A* 的流程对应框架中的 `core.solver.algorithm.searcher.BestFirstSearcher`：

```
Open ← {初始结点}；Closed ← ∅
while Open 非空:
    n ← Open 中 f 值最小的结点
    若 n 是目标状态：沿 parent 指针回溯输出解路径，结束
    把 n 的状态放进 Closed
    for each 后继结点 m of n:
        若 m 的状态已在 Closed 中：舍弃
        否则:
            若 m 的状态已在 Open 中:
                保留两者中 g 值较小的那个（本框架通过 Frontier 的 offer 契约实现）
            否则：算出 f(m) 并放进 Open
返回无解
```

1.2 算法的可采纳性

1.2.1 A* 算法

可采纳性（admissible）的定义：对任意结点 n 都有

$$0 \le h(n) \le h^*(n)$$

其中 $h^*(n)$ 是从 n 到目标的真实最小代价。说白了，h 可以低估，但不能高估，高估就不再保证最优了。

单调性（consistent）的定义：对任意结点 n 和它的任一后继 n'（转移代价记为 $c(n,n')$）都有

$$h(n) \le c(n,n') + h(n')$$

意思是沿着路径往前走，f 值不会往下掉。

1.2.2 A* 算法可采纳性证明

定理：若 h 可采纳，则 A* 把目标状态取出来扩展时，它的路径代价 g 就是最优代价 C^*。

证明：先证一个引理。

引理：在 A* 终止之前的任意时刻，最优路径上都存在一个结点 n 已经躺在 Open 表里，且 $g(n) = g^*(n)$。

引理证明：设最优路径为 $n_0 \to n_1 \to \cdots \to n_k$（$n_0$ 初始，n_k 目标）。初始时 n_0 在 Open 里且 $g(n_0) = g^*(n_0) = 0$，引理成立。

假设某时刻引理成立且算法还没停，此时 Open 里满足条件的那个路径结点记为 n_i：

1. 若 n_i 不是目标，算法会挑某个结点扩展。挑的不是 n_i 时，n_i 仍留在 Open 里，引理继续成立；挑的正是 n_i 时，n_{i+1} 会被生成——如果它已经在 Open 里，算法会保留 g 更小的那个，于是 $g(n_{i+1}) = g^*(n_{i+1})$；如果不在，就以 $g(n_{i+1}) = g(n_i) + c(n_i,n_{i+1}) = g^*(n_{i+1})$ 放进去。两种情况都保证 n_{i+1} 在 Open 里且 g 最优。
2. 若 n_i 就是目标 n_k，算法立刻终止，不在讨论范围内。

引理证毕。

回到定理，用反证法。假设 A* 取出目标结点 t 时 $g(t) > C^*$。

由引理，此刻最优路径上有个结点 n 在 Open 里且 $g(n) = g^*(n)$，于是

$$f(n) = g(n) + h(n) = g^*(n) + h(n) \le g^*(n) + h^*(n) = C^* \quad (1)$$

这里用到了 h 可采纳。

又因为 A* 每次都取 f 最小的结点，它取了 t，所以

$$f(t) \le f(n) \quad (2)$$

目标结点的 $h(t) = 0$，故 $f(t) = g(t)$。把 (1)(2) 合起来得 $g(t) \le C^*$。

但 $g(t)$ 是某条真实路径的代价，而 C^* 是所有路径里最小的那个，所以 $g(t) \ge C^*$。

两边一夹，$g(t) = C^*$，与假设矛盾。定理证毕。

顺带说一句：这个证明只用到可采纳性，**没有用到单调性**。单调性是 1.3.2 讨论"能不

能不重新开放结点"时才需要的。

1.3 重复扩展结点的问题

1.3.1 重复扩展结点问题

N-Puzzle 属于图搜索问题，同一个状态往往有多条路径能走到（比如空格绕一圈回到原处）。不处理的话，同一个状态会被反复生成、甚至反复扩展，后果是 Open/Closed 表疯狂膨胀、大量重复计算，严重时程序根本收敛不了。

本框架的处理方式在 `BestFirstSearcher.search` 里：

1. 扩展结点 n 时，把 n 的**状态**放进 Closed（`explored`，类型是 `Set<State>`）；
2. 生成后继 m 时，如果 m 的状态已经在 Closed 里，直接扔掉；
3. 否则交给 Frontier 的 `offer`——如果 Open 里已经有同状态结点，**保留两者中较好的那个**（这是 `core.solver.queue.Frontier` 接口定下的契约）。

需要留意的是，本框架采取的是"状态一旦进 Closed 就不再回 Open"的策略。这种做法只有在 h 单调时才能保证最优。如果 h 只有可采纳性、没有单调性，正确做法是允许把 Closed 中的结点重新放回 Open。

1.3.2 启发函数的单调性

定理：若 h 单调，则 A* 第一次扩展某状态 s 时拿到的 $g(s)$ 已经是该状态的最优代价 $g^*(s)$，所以不需要重新开放结点。

证明（要点）：设 s 第一次被扩展时 $g(s) > g^*(s)$，最优路径上 s 的前驱记为 n'。当 n' 被扩展时，算法会以 $g^*(s)$ 更新或插入 s。由于 h 单调，沿最优路径的 f 值单调不降，可以推出 s 会先以 $g^*(s)$ 被取出扩展，矛盾。故首次扩展时的 g 就是最优值。

由单调性还能顺手得到两条结论：

1. **单调蕴含可采纳**（当 $h(\text{目标}) = 0$ 时）：沿着路径反复套用 $h(n) \le c(n,n') + h(n')$ 累加起来，就是 $h(n) \le h^*(n)$；
2. 单调时 f 沿路径不降，Open 里被剪掉的结点不用再回头看，实现起来更省事也更快。

这里要老实交代一件事：我们实现的三个启发函数里，曼哈顿距离和不在位将牌数都是单调的（2.4 节给了证明）。第三阶段的模式数据库，我们**严格证明了它的可采纳性**，但**没能给出单调性的严格证明**。

框架的 A* 用的又是"不重新开放"策略，理论上就存在拿到次优解的风险。我们的应对办法是拿实验兜底：在所有测试实例上，启用模式数据库的 A* 给出的解路径长度，与两个已经证明单调的可采纳启发函数（曼哈顿、不在位将牌）以及 IDA*（只要求可采纳性）的结果**完全一样**，没有观察到任何次优的情况。

1.4 IDA* 算法基本原理及其可采纳性

A* 有个绕不开的负担：它得把生成过的结点全留在内存里。15 数码的状态空间大约 10^{13}，困难实例下 A* 的内存占用能到几个 GB。IDA*（Iterative Deepening A*）的思路是**拿**

时间换空间——它一张 Open 表都不存，只留当前这一条搜索路径。

基本原理：对 f 值做迭代加深。

```
threshold ← h(初始结点)          // 起点 g = 0，所以 f = h
loop:
    t ← 以 threshold 为界做一次深度优先搜索
    若找到目标：返回解路径
    若 t = ∞ ： 返回无解
    threshold ← t                 // 抬高阈值，重新搜索
```

内层深度优先搜索的规则是：

- 若 $f(n) > \text{threshold}$，剪枝，并把这个 f 值返回给上一层（下一轮的阈值要从这些值里取最小）；
- 若 n 是目标，找到解；
- 否则做一次环检测后递归扩展子结点；
- 最后返回"本轮所有被剪掉分支中最小的 f 值"。

可采纳性与最优性：h 可采纳时，每一轮的 threshold 都是个下界，第一轮找到解时 threshold 恰好等于 C^*，所以解一定最优。直观理解就是：threshold 会依次取遍所有可能的 f 值，而目标结点的 f 值就是它的路径代价，因此第一次撞上目标时的 threshold 就是最优代价。

两者对比：

	A*	**IDA***
内存	O(已生成结点数)，可达数 GB	O(解路径长度)，通常只有几 KB
时间	每个状态只扩展一次	每抬高一次阈值，浅层就要**重搜一遍**，结点数更多
最优性	h 可采纳即最优	h 可采纳即最优
适用场景	内存充足	**内存受限、解路径不太长**（15 数码的经典选择）

我们自己的实验就是这样：作业第三阶段那个倒序实例，A* 配模式数据库**吃满 12 GB 内存还是 OutOfMemory**；换成同样启发函数的 IDA*，只占 516 MB 就做完了。反过来，3 阶实例上 IDA* 反而略慢于 A*，因为实例太小、启发式剪枝省不下多少，重复搜索的开销倒是实打实的。

2. N-Puzzle 游戏

2.1 N-Puzzle 游戏描述

N-Puzzle 就是数字华容道：一个 $n \times n$ 的方格盘，里面放着 n^2-1 块编号 $1 \sim n^2-1$ 的可滑动将牌，外加 1 个空格。每次只能把**空格**和它上下左右相邻的将牌对调。目标是把盘面恢复成规定的标准排列（本项目中是行优先的 $1,2,3,\ldots,n^2-1,0$，其中 0 表示空格）。

两个规模上的事实：3 阶（8 数码）任何有解盘面都能在 **31 步**内解决；4 阶（15 数

码）最坏情况需要 **80 步**。

2.2 游戏的建模、状态及状态空间、状态转换规则

按框架的抽象，N-Puzzle 可以拆成四块：

概念	框架抽象类	本项目实现	说明
状态	`core.problem.State`	`PuzzleBoard`	一个 $n \times n$ 盘面，0 表示空格
动作	`core.problem.Action`	`Move`	空格朝某方向移动一格
问题	`core.problem.Problem`	`NPuzzleProblem`	初始状态 + 目标状态 + 规模
结点	`core.solver.queue.Node`	（框架提供）	状态 + 父结点 + 动作 + g + h

状态空间规模：$n \times n$ 的盘面上，将牌和空格的排列一共 $(n^2)!$ 种，但可达状态只占一半（原因见 2.3），所以

- 3 阶：$9!/2 = 181{,}440$ 个可达状态（我们用 BFS 把这一整个空间遍历过，数字对得上）；
- 4 阶：$16!/2 \approx 1.05 \times 10^{13}$ 个可达状态。

状态转换规则：设空格在 (r, c)，方向 d 的行列偏移是 $(\Delta r, \Delta c)$，则

$$\text{next}(s, d) = \text{swap}(s[r][c], s[r+\Delta r][c+\Delta c]), \quad \text{当且仅当 } (r+\Delta r, c+\Delta c) \text{ 落在盘面内}$$

也就是说，**一个动作能不能做，完全取决于空格是不是贴着边界**。本项目里每个动作的代价恒为 1，所以 g 就等于已经走了多少步。

2.3 N-Puzzle 可解性研究

N-Puzzle 的状态空间被切成**两个互不连通的连通分量**，从任一状态出发只能到达它所在分量里的状态。判断依据是**逆序数的奇偶性**。

逆序数：把盘面上的数字按行优先排成序列（**去掉空格**），序列中满足 $i < j$ 且 $a_i > a_j$ 的数对个数就是逆序数 N。

判定规则（实现在 `NPuzzleProblem.solvable()`）：

- **奇数阶**（比如 3×3）：N 为**偶数**时有解；
- **偶数阶**（比如 4×4）：记 b 为**空格"从下往上数"的行号**（最下面一行是 1），则

$$(N + b) \text{ 为奇数} \Leftrightarrow \text{有解}$$

偶数阶为什么必须多这一条？因为每一步滑动都会让空格在行方向上动一下，同时改变逆序数的奇偶性，两个变化合在一起才构成不变量。

实现上做了一点小处理：为了让目标盘面可以是任意排列（而不是只能是最标准的 $1,2,3,\ldots,0$），我们统计逆序数时是**以目标盘面的先后次序为基准**的——先把初始盘面的每块将牌映射到它在目标盘面中的位置，再统计这个序列的逆序数。这样对任意合法的目标盘面都成立。

验证：我们构造了 9 组测试用例（3 阶和 4 阶各有"有解"和"无解"的实例，比如把目

标盘面里相邻两块将牌对调），**9 组全部判断正确**。

2.4 三个启发函数的有效性和单调性证明

设盘面边长为 n，将牌 t 现在在 (r_t, c_t)，它在目标盘面里应该在 (r_t^*, c_t^*)。

2.4.1 曼哈顿距离

$$h_{\text{man}}(s) = \sum_{t \neq 0} (|r_t - r_t^*| + |c_t - c_t^*|)$$

就是所有将牌（**不含空格**）当前位置到目标位置的曼哈顿距离之和。

可采纳性：每走一步只有一块将牌移动一格。所以每块将牌至少要挪 $|r_t - r_t^*| + |c_t - c_t^*|$ 步才能回家，全部将牌所需的步数至少是它们各自步数之和，于是 $h_{\text{man}}(s) \leq h^*(s)$。

这里有个必须注意的点：**空格要排除在外**。空格移动只是将牌移动的副产品，把它也算进去就会高估真实代价，可采纳性就没了。代码里是显式跳过空格的。

单调性：设从 s 执行动作 a 得到 s'，代价 $c(s,a) = 1$。这一步只会让**一块**将牌 t_0 动一格：

- 如果这一步让 t_0 更接近目标，则 $h_{\text{man}}(s') = h_{\text{man}}(s) - 1$，即 $h(s) = h(s') + 1 = c + h(s')$；
- 如果这一步让 t_0 远离目标，则 $h_{\text{man}}(s') = h_{\text{man}}(s) + 1$，此时 $h(s) < h(s') + 1 = c + h(s')$。

两种情况都满足 $h(s) \leq c(s,a) + h(s')$，所以曼哈顿距离单调。

2.4.2 不在位将牌（错位将牌数）

$$h_{\text{mis}}(s) = |\{\ t \neq 0 : (r_t, c_t) \neq (r_t^*, c_t^*)\ \}|$$

就是位置不正确的将牌**块数**（空格不算）。

可采纳性：每走一步最多让一块将牌归位，所以至少还要走"不在位将牌数"这么多步，$h_{\text{mis}}(s) \leq h^*(s)$。

单调性：一步最多让不在位将牌数减少 1。减少了就是 $h(s) = h(s') + 1 = c + h(s')$，没减少则 $h(s) \leq h(s') + 1$，两种情况都满足单调条件。

还有一个关系值得记一下：每块不在位的将牌，它的曼哈顿距离至少是 1，所以

$$h_{\text{mis}}(s) \leq h_{\text{man}}(s)$$

也就是说，**不在位将牌数永远不会比曼哈顿距离更紧**。实测性能也差得远，见 4.3。

2.4.3 Disjoint Pattern（不相交模式数据库）

思路是：把 15 块将牌分成若干**互不相交**的子集（每个子集叫一个"模式"），对每个模式单独算出"把它还原需要多少步"，结果存成一张查找表（模式数据库，Pattern Database），搜索时查表并**相加**。

抽象状态：对模式 P，它的抽象状态由三部分构成——模式内每块将牌所在的格子，以及**空格**所在的格子。模式之外的将牌一律看成**互相无法区分的"其他块"**。空格必须放进来，

因为盘面本来就是靠"空格和相邻将牌对调"演化的。

状态总数：把 k 块模式将牌和 1 个空格放进 n^2 个格子（各不相同），共有

$$n^2 \times (n^2-1) \times \cdots \times (n^2-k)$$

种抽象状态。4 阶取 $k=5$ 时是 $16\times15\times14\times13\times12\times11 = 5{,}765{,}760$。

代价划分（cost partitioning）

如果对每个抽象动作都老老实实计 1 步，再把各模式的值加起来，会**高估**真实代价，可采纳性就丢了。用一个 2×2 的小例子就能看穿：

目标盘面 [1,2,3,_]，*状态* [1,2,_,3]，*真实距离是 1（空格右移一格就完事）。*

取模式 {1},{2},{3}：

模式 {1} *要把空格从第 2 格挪到第 3 格，1 步；*

模式 {2} *同理，1 步；*

模式 {3} *要把将牌 3 从第 3 格移到第 2 格，1 步。*

加起来是 3，比真实距离 1 还大，这显然不能用。

正确做法是定一条规则：

只有"移动本模式内的将牌"才计 1 步；空格穿过非本模式将牌（也就是移动别的将牌）一律计 0 步。

可采纳性证明：设真实最优解有 L 步。每一步滑动一块将牌，而每块将牌恰好属于一个模式，把这一步"记账"到它所属的模式上，那么各模式的记账步数 c_P 之和正好是 L。对模式 P，把整个解投影到 P 的抽象空间，就得到一条代价不超过 c_P 的抽象路径，所以

$$d_P(s) \le c_P$$

对所有模式求和：

$$\sum_P d_P(s) \le \sum_P c_P = L$$

相加式模式数据库启发函数确实可采纳。

它一定不弱于曼哈顿距离：按代价划分的规则，模式内每块将牌每次"被计数"的移动最多让它离目标近一格，所以 d_P 不小于该模式内各将牌的曼哈顿距离之和。把所有模式加起来正好覆盖全部将牌，于是

$$\sum_P d_P(s) \ge h_{\text{man}}(s)$$

换句话说，这个启发函数在任何盘面上都不会比曼哈顿距离差。

取最大值仍然可采纳：本项目实际使用的是

$$h_{\text{pdb}}(s) = \max\left(h_{\text{man}}(s) + 2 \times \text{冲突对数},\ \sum_P d_P(s) \right)$$

可采纳性的要求是"估计值不超过真实剩余步数"。线性冲突和模式数据库相加值各自都满足，而**若干个不超过 L 的数取最大，结果也不会超过 L**，所以取 max 是安全的。

顺便说一下线性冲突的来由：同一行里如果有两块将牌**都已经在自己的目标行上**、但左右顺序反了，那它们得"绕一圈"才能换过来——至少多花 2 步（一块先让开、另一块过去、它再回来）。列方向同理。所以 $h_{\text{man}} + 2\times$冲突对数 仍然是下界。代码在 `PuzzleBoard.linearConflict()`。

3. N-Puzzle 求解算法实现

3.1 状态及结点的数据结构

状态：PuzzleBoard，用 64 位整数把盘面打包起来

N-Puzzle 每格取值 $0 \sim 15$，正好 4 个二进制位；4 阶有 16 格，$16 \times 4 = 64$ 位，**刚好塞得进一个 long：**

格 0 存在第 0~3 位，格 1 存在第 4~7 位，……，格 i 存在第 4i ~ 4i+3 位

例：[8,6,7, 2,5,4, 3,0,1] → bits = 8 | 6<<4 | 7<<8 | 2<<12 | …

这么做带来的差别相当大：

对比项	用 int[] 数组（朴素写法）	用 long 打包（本项目）
单个状态内存	对象约 40 B + int[16] 约 80 B ≈ **128 B**	单对象约 **48 B**
equals	Arrays.equals，O(16)	**一次 long 比较**，O(1)
hashCode	每次遍历 16 个元素	构造时算好缓存，**O(1)**
next()	复制数组再交换	**两个 4 位段对调，一个减法**，不分配数组
附带好处	必须 clone 外部数组防篡改	数据已经"烤"进 long 里，天然不可变

next() 的做法是：设目标格上的将牌为 v，则

$$\text{bits}' = \text{bits} + (v \ll 4 \cdot \text{blank}) - (v \ll 4 \cdot \text{target})$$

加减都刚好发生在各自的 4 位段内部（结果分别是 v 和 0，都落在 0~15 之内），不会产生进位或借位。

结点：Node（框架提供）

框架的 Node 持有状态、父结点、动作、g（pathCost）和 h（heuristic），并提供

$$\text{evaluation}() = g + h$$

以及三种比较器（对应 EvaluationType 的三种估值策略）。它的 equals 只比较**状态**，所以"同状态不同路径"的结点会被当成同一个。

我们在代码审查时发现框架的 Node 重写了 equals 却**没有重写 hashCode**，这违反了 Java 约定。好在我们的 Open 表索引用的是 Map<State, Node>（键是 State 不是 Node），Closed 表用的是 Set<State>，都没有拿 Node 当哈希键，所以这个缺陷对我们没有实际影响。框架文件按作业要求不作修改。

3.2 Open 表及 Closed 表的数据结构

Closed 表：框架的 explored

类型是 Set<State>（HashSet）。去重全靠 PuzzleBoard 正确实现了 equals/hashCode——这也正是 3.1 节打包优化收益最大的地方。IDA* 每生成一个子结点都要查一次"是不是已经在当前路径上"，把原来 O(16) 的数组哈希换成 O(1) 的缓存哈希之后，**同一个倒序实例的 IDA* 耗时从 325 秒降到 108 秒**（见 4.2）。

Open 表：PqFrontier，二叉堆加索引，再配惰性删除和定期压实

PqFrontier 继承 PriorityQueue<Node> 并实现 Frontier 接口。做下来一共撞上三个难点。

第一：优先队列不支持按状态查找

框架要求 offer 遵守契约：Open 里如果已经有同状态的结点，就丢掉两者中较差的。可 PriorityQueue 只保证堆顶最小，想知道某个状态在不在堆里，只能老老实实线性扫一遍，复杂度 $O(N)$。15 数码上搜索会生成上千万个结点，而"生成一个已经见过的状态"是家常便饭，这样一来总计算量会飙到 10^{11} 量级。

解决办法是额外维护一张 Map<State, Node> 索引表，把查找降到 $O(1)$。

第二：怎么把旧结点换掉

第一版写法是发现更优结点后调用 PriorityQueue.remove(oldNode) 把旧的摘掉。可 PriorityQueue 内部并没有"元素到位置"的索引，remove 还是得线性扫整个堆，照样是 $O(N)$。

改用**惰性删除**：不去堆里删旧的，而是

1. 把索引表里该状态的条目**覆盖**成新结点；
2. 新结点照常入堆，**旧结点就留在堆里当"废结点"**；
3. poll() 取到结点时，用**身份比较**（==）判断它是不是该状态当前的最优——不是就丢掉，继续取下一个。

这样 offer 和 poll 都只有 $O(\log N)$，删旧的成本被摊还到了 poll 里。

由此带出一个容易忽略的坑：isEmpty() / size() 必须一起重写

惰性删除之后，堆里可能**只剩下一堆废结点**。这时候如果还用 PriorityQueue.isEmpty()（它看的是堆数组里有没有元素），会返回 false，框架就以为还有结点可取、调用 poll()；而 poll() 把废结点全过滤掉之后只能返回 null，紧接着框架执行 problem.goal(node.getState()) 就**空指针崩了**。

所以本项目把 isEmpty()/size() 重新定义为"**看索引表**"：索引表里有的才算存活结点，这个数量也正好就是 Open 表里真正能取的结点数。

第三：废结点把内存撑爆（这个是实测撞上的）

惰性删除的代价是"废结点只进不出"：每替换一次就多留一个永远取不到、也永远被 GC 收不掉的旧结点。在困难实例上这是要命的——我们实测在倒序实例上跑到第 272 秒时，程序在 PriorityQueue.grow() 里 OutOfMemory 了。

加**定期压实**解决：当"堆内元素总数"超过"存活结点数 × 2 + 1024"时，把存活结点抓出来、清空堆、再放回去，废结点随即被 GC 回收。两次压实之间至少会插入 k 个结点（k 是存活数），所以摊还下来每次 offer 仍然只有 $O(\log N)$，而堆的内存有了上界。

修完之后重跑，OOM 的位置从"废结点撑爆堆数组"变成了"存活结点本身超过 12 GB"——瓶颈从数据结构缺陷转移到了算法固有复杂度上。

3.3 三个启发函数的具体实现

3.3.1 曼哈顿距离

实现在 PuzzleBoard.manhattan(PuzzleBoard goal)，有两个性能上的讲究：

1. **预计算"数值到下标"的查找表**：把目标盘面的 pos[将牌编号] = 下标 缓存起来（惰性构建），每次估值就从 $O(n^2)$ 降到 $O(n)$；
2. **预计算"格 i 到格 j"的距离表**：曼哈顿距离里的除法和取模挺贵，干脆把 $n^2 \times n^2$ 的距离矩阵事先算好（4 阶也就 256 个 int），运行时就是一次数组读取。

```
public int manhattan(PuzzleBoard goal) {
    int[] goalPos = goal.positionIndex();      // pos[将牌编号] = 它在目标中的下标
    int[] distance = distanceTable(size);      // 预计算好的距离表
    int sum = 0;
    for (int i = 0; i < n; i++) {
        int v = (int) ((bits >>> (4 * i)) & 0xF);
        if (v == 0) continue;                  // 空格不计入
        sum += distance[i * n + goalPos[v]];
    }
    return sum;
}
```

3.3.2 不在位将牌

同样是遍历 16 个 4 位段，统计"值不为 0 且与目标不同"的块数，实现结构和曼哈顿距离一模一样，只是判断条件不同。

3.3.3 Disjoint Pattern 和 Pattern Database

实现在 PatternDatabase.java（约 600 行）。四个关键机制：

模式怎么分（patternSetsFor）

4 阶默认用 **5-5-5**：{1,2,3,4,5}、{6,7,8,9,10}、{11,12,13,14,15}。三组互不相交，满足相加式启发函数的前提；每组 5 块时抽象状态是 576 万，建表和内存都还能接受。

3 阶必须另外给划分（$\{1,2,3\},\{4,5,6\},\{7,8\}$）——因为 4 阶那套里的 $\{11,\ldots,15\}$ 在 3 阶盘面上**一块将牌都不存在**，查表会查到毫无意义的数值。

完美哈希（状态排名）（rankOf）

把 $k+1$ 个对象（k 块模式将牌加 1 个空格）依次摆放，用混合进制算排名：

```
rank = 0;  mask = 0（已占用格子的位掩码）
对第 i 个对象 (i = 0..k):
    p = 该对象所在格子
```

```
它前面还有多少"空着的格子" = p - popcount(mask 中 < p 的位)
rank = rank × (n² - i) + 上面的值
mask 置上第 p 位
```

每一步的选择数依次是 $n^2, n^2-1, \ldots, n^2-k$，所以 rank 恰好落在 $[0, \text{stateCount})$ 之内，**没有空位、没有冲突**，可以直接当数组下标用。

反向建表：分层 0-1 BFS（build）

因为有 0 代价边，普通 BFS 就不对了。本实现按"距离层"推进，每一层做两件事：

1. **零代价闭包**：从本层状态出发，沿"0 代价边"（空格穿过其他将牌）不断扩散，把新状态并进本层（用同一层列表动态增长来实现）；
2. **产生下一层**：等本层的闭包铺完了，再沿"1 代价边"（移动模式内将牌）产生下一层的状态。

这个顺序不能反。必须先把本层铺满再去产生下一层，否则本来该在 d 层的状态会被误判成 $d+1$ 层，启发值偏大，**可采纳性就破了**。

组合启发函数（combined）

```
public static int combined(PuzzleBoard state, PuzzleBoard goal) {
    int best = state.linearConflict(goal);          // 线性冲突：不需要建表
    for (PatternDatabase[] set : forGoal(goal)) {   // 各套模式数据库的相加值
        int sum = sumOf(set, state);
        if (sum > best) best = sum;
    }
    return best;
}
```

4 阶 5-5-5 的实测开销：每组 5,765,760 个抽象状态，**全部访问到**，单组建表约 0.7 秒，三组加起来约 **2.3 秒**，距离表共 **33 MB**。

建表时间之所以可以忽略，是因为它是**一次性预处理**，之后所有实例共用。以阶段二为例，2.3 秒的建表换来的是后续每个实例从 0.5~110 秒降到 0.07~1.05 秒，这笔账怎么算都划算。

3.4 求解后继结点的函数

框架把"生成后继结点"写成了 Problem 上的 final 模板方法 childNodes(Node parent, Predictor predictor)，流程是：

```
for each action ∈ parentState.actions():        ← PuzzleBoard.actions()
    若 applicable(parentState, action):         ← NPuzzleProblem.applicable()
        state ← parentState.next(action)        ← PuzzleBoard.next()
        pathCost ← parent.getPathCost() + stepCost(state, action)   ← NPuzzleProblem.stepCost()
        child ← new Node(state, parent, action, pathCost)
        child.setHeuristic(predictor.heuristics(state, goal))       ← 启发函数
        收集 child
```

我们填的坑对应如下：

环节	实现位置	说明

actions()	PuzzleBoard	返回 4 个方向的 Move（**共享实例**，避免每次扩展都 new 对象）
applicable()	NPuzzleProblem	空格朝该方向移动后是否越界
next()	PuzzleBoard	两个 4 位段对调（见 3.1）
stepCost()	NPuzzleProblem	恒为 1
heuristics()	PuzzleBoard / PatternDatabase	三个启发函数通过 EnumMap<HeuristicType, Predictor> 注册

动作类还做了实例缓存：N-Puzzle 一共就 4 种动作，而且动作不可变（只包了一个方向枚举），那就干脆预先造好 4 个实例反复用。搜索会产生几千万甚至上亿个结点，这个看起来不起眼的优化能省掉大量垃圾对象。

4. N-Puzzle 问题求解探讨（实验数据分析）

实验环境：Intel Core Ultra 5 125H（14 核 18 线程）加 31.6 GB 内存；老师给的参考结果跑在 **Dell OptiPlex 3080** 上。两台机器不一样，所以时间只能做**同量级**的比较。真正与机器无关的硬指标是**解路径长度**和**扩展结点数**，下面的对比重点看这两项。运行时都显式指定了堆大小；计时里包含一次性建表时间的地方会单独标注。

4.1 Open 表及 Closed 表数据结构的改进及其对算法性能的影响

阶段二（4 阶）我们做了三轮递进优化，每一轮都留了量化对比：

实例	**① 基线**	**② 惰性删除**	**③ long 打包**	**④ 惰性删除 + 压实**	**相对基线**
实例 1（52 步）	0.728 s	0.843 s	0.475 s	0.475 s	1.5 倍
实例 2（51 步）	15.502 s	13.233 s	6.476 s	6.476 s	2.4 倍
实例 3（56 步）	49.989 s	47.752 s	21.506 s	21.506 s	2.3 倍

这里有几个结论跟直觉不太一样：

1. **惰性删除（②）单独只贡献了 4%~14%**，说明 $O(N)$ 替换并不是主要矛盾。我们一开始认定它是性能杀手，实测直接把这个判断推翻了。
2. **long 打包（③）才是大头**。它一次改善了三件事：内存减半、equals/hashCode 从 $O(16)$ 变 $O(1)$、省掉几千万次数组分配和随之而来的 GC。
3. **正确性上没问题**：三轮优化前后的**结点数逐位一致**（比如实例 1 的扩展结点恒为 339,410），说明搜索行为没有被改动，解仍然最优。

同一个 long 打包，放到 IDA* 上收益更大：老师给的 9 个实例，我们的 IDA* 总耗时 136.2 秒，参考程序是 404.1 秒，约 3 倍（逐项对标见 4.2.1）。原因是 IDA* 每生成一个子结点都要查一次"是不是在当前路径上"，long 打包把这步从 $O(16)$ 降到 $O(1)$；而 A* 的时间大头花在堆操作的 $O(\log N)$ sift 上，那部分我们和参考实现一样。

4.2 A *算法*、*IDA* 算法的性能比较

4.2.1 与参考程序的逐项对标

先看 A*（老师提供 6 个实例）：

实例	参考解长	我们解长	参考耗时（两次）	我们耗时	参考扩展	我们扩展
1（3 阶）	21	21	0.0156 / 0.0156 s	0.004 s	931	931
2（3 阶）	31	31	0.0938 / 0.0781 s	0.033 s	21,197	21,197
3（3 阶）	31	31	0.0469 / 0.0313 s	0.014 s	21,197	21,197
4（4 阶）	52	52	0.7969 / 0.7344 s	0.516 s	339,410	339,410
5（4 阶）	51	51	9.2188 / 9.0469 s	8.039 s	3,628,148	3,628,148
6（4 阶）	50	50	11.6406 / 11.3906 s	12.667 s	5,229,940	5,229,940

解路径长度 6 个全对，扩展结点数 6 个**逐位相同**。这比"时间差不多"能说明更多问题：扩展结点数一样，意味着我们的搜索行为和参考程序是等价的（同样的启发函数、同样的去重规则、同样的取结点顺序）。耗时 5 项更快、1 项略慢，考虑到机器差异，可以认为在同一水平线上。

再看 IDA*（9 个实例）：

实例	参考解长	我们解长	参考耗时	我们耗时	加速比
1–3（3 阶）	21 / 31 / 31	一致	0.078 s	0.023 s	约 3 倍
4（4 阶）	52	52	0.734 s	0.216 s	3.4 倍
5（4 阶）	51	51	1.750 s	0.521 s	3.4 倍
6（4 阶）	56	56	5.969 s	5.764 s	1.0 倍
7（4 阶）	57	57	65.313 s	20.905 s	3.1 倍
8（4 阶）	50	50	3.578 s	1.723 s	2.1 倍
9（4 阶）	61	61	325.844 s	108.070 s	3.0 倍
合计			404.1 s	136.2 s	约 3.0 倍

解路径长度 9 个全对，说明 IDA* 实现正确、而且同样返回最优解。

4.2.2 A 与 IDA 的对比分析

IDA* 在本题上常常比 A* 更快

拿参考数据里同一台机器的结果对比：

实例	A* 耗时	IDA* 耗时	谁快
4 阶实例 4（52 步）	0.797 s	0.734 s	IDA*
4 阶实例 5（51 步）	9.219 s	1.750 s	IDA* 快 5.3 倍
4 阶实例 8（50 步）	11.641 s	3.578 s	IDA* 快 3.3 倍

原因在于 IDA* 没有堆操作（$O(\log N)$ 的 sift-up/down），也没有哈希表查询，**常数因子小得多**。只要启发函数足够强、重复搜索的开销不占主导，这个常数优势就压过了"重搜"的劣势。

但 A* 的内存是致命瓶颈

方案	倒序实例的结果
A* 配模式数据库	12 GB 堆还是 OutOfMemory（死在 Open 表索引扩容）
IDA* 配同一启发函数	只在 516 MB 内就做完了

内存差了 23 倍以上。这就是 1.4 节说的"内存换时间"的直接证据，也解释了为什么经典 15 数码求解器都选 IDA*。

4.3 三个不同的启发函数及 IDA* 的性能比较

4.3.1 运行时间对比及分析

实验一：3 阶（阶段一要求的实例，搜索用 A*）

实例	不在位将牌数	曼哈顿距离	模式数据库
1（21 步）	0.016 s	0.000 s	0.019 s
2（31 步）	0.156 s	0.016 s	0.019 s
3（31 步）	0.141 s	0.016 s	0.022 s

3 阶规模太小，4 毫秒左右的建表时间反倒成了主要开销，所以模式数据库在这里并不占便宜。不过作业要求是"1 秒内解出"，三个启发函数都轻松达标，余量在 6 倍以上。

实验二：4 阶（阶段二要求的实例，搜索用 A*）

实例	解长	不在位将牌数	曼哈顿距离	模式数据库	PDB 相对曼哈顿
1	52	OutOfMemory	0.475 s	3.800 s（含建表 3.59 s）	建表开销占主导
2	51	OutOfMemory	6.476 s	0.264 s	24.5 倍
3	56	OutOfMemory	21.506 s	0.289 s	74.4 倍
4	57	OutOfMemory	110.550 s	1.049 s	105 倍
5	50	OutOfMemory	13.191 s	0.069 s	191 倍

6	61	OutOfMemory	跑不动	31.522 s	—
7（倒序）	70	OutOfMemory	OutOfMemory	OutOfMemory（见 4.3.4）	—

三条结论：

1. **"不在位将牌数"在 4 阶上基本没法用**。它的估计太松，A* 需要扩展天文数字的结点，7 个实例全部 OutOfMemory。这不是我们代码的问题，而是这个启发函数强度的固有限制。
2. **曼哈顿距离能用但不够**，最困难的两个实例一个要 110 秒、一个直接 OOM。
3. **模式数据库快 1 到 2 个数量级**，还把实例 6 从"跑不动"变成"31 秒解出"。这也说明阶段二真正的瓶颈在启发函数上，不是数据结构：数据结构最多带来 2.4 倍，换启发函数就是 100 倍以上，两者不在一个量级。

上表里模式数据库用的是 **5-5-5 纯相加式**的版本；实例 1 的 3.800 秒包含首次建表的 3.59 秒，后面几个实例共用建好的表，所以从实例 2 起才是纯搜索时间。

实验三：启发函数取值对比（同一批实例，看初始状态）

为了解释上面的差异，我们直接把各启发函数在初始状态上的取值测了出来：

实例（最优步数）	曼哈顿	线性冲突	5-5-5	行 4-4-4-3	列 4-4-4-3	6-6-3	取最大值
倒序实例（70）	44	56	52	50	56	45	56
57 步实例	41	43	47	43	41	—	47
50 步实例	34	36	42	40	36	—	42
52 步实例	44	44	44	44	44	—	44

这张表能解释三件事：

1. **"不在位将牌数"为什么不如曼哈顿**：$h_{\text{mis}} \le h_{\text{man}}$ 是恒成立的（每块不在位将牌的曼哈顿距离至少是 1），本质上就是"块数"和"距离"的差别。
2. **线性冲突的性价比极高**。它**一张表都不用建**（纯算术、零内存），在倒序实例上却给出 56，比精心建了 33 MB 查找表的 5-5-5（52）还高 4 分。道理也不复杂：倒序盘面的行和列几乎都是反的，线性冲突一眼就能看出"这些将牌必须多绕路"。
3. **模式组不是越大越强**，6-6-3 只有 45，比 5-5-5 的 47 还低。下一节细说。

4.3.2 扩展结点比较及分析

实验四：扩展结点数对比（4 阶，A*）

实例	解长	曼哈顿扩展	PDB 扩展	降幅
1	52	339,410	50,111	-85%
2	51	3,628,148	76,657	-98%
3	56	10,464,927	105,370	-99%
4	57	33,017,378	316,107	-99%
5	50	5,229,940	31,900	-99%
6	61	跑不动	9,590,506	—

结点数的降幅（98%~99%）比时间的降幅（24~191 倍）还要夸张，原因是模式数据库单次估值更贵：每个模式都要扫一遍盘面再做排名计算。所以看一个启发函数不能只盯着它压掉多少结点，还要看单次估值贵了多少，两头乘起来才是真实收益。

我们自己就吃过这个亏。第一版"组合启发函数"把三套划分加上线性冲突全部塞进 max：扩展结点少了 21%，但**耗时反而从 2.654 秒涨到 3.404 秒——**原因是线性冲突里对每个冲突对都做了除法和取模。把"每块将牌的目标行、目标列"**预计算成小表**之后，耗时回到 **2.812 秒，只比纯 5-5-5 慢 6%**，而扩展结点还少了 6%。

4.3.3 模式组并非越大越强

阶段三我们试过把 5-5-5 换成每组更大的 **6-6-3**，想法很自然：组更大，组内阻挡信息更完整，启发值应该更强。实测的结果有点出乎意料：

对比项	5-5-5	6-6-3
单组抽象状态数	5,765,760	57,657,600（10 倍）
距离表大小	33 MB	220 MB（6.7 倍）
建表总耗时	2.3 s	37.5 s（16 倍）
57 步实例的启发值	47	45（反而更低）
57 步实例的扩展结点	1,105,287	4,434,872（4 倍）
57 步实例的耗时	2.812 s	38.448 s（慢 13.7 倍）

开销大了十几倍，效果反而更差。原因在于：**代价划分模型下，模式组的大小不是唯一变量，划分得均不均衡同样关键**。6-6-3 里第三组只有 {13,14,15} 三块，贡献太小；5-5-5 三组均衡，组内的阻挡信息利用得更充分。而组越大，建表和内存的代价是成倍往上涨的，却换不来启发值。

4.3.4 阶段三倒序实例的求解

PDF 第三阶段要求 **1 分钟内解出**指定的倒序实例：

```
15 14 13 12
11 10  9  8
 7  6  5  4
 3  1  2  _
```

我们的求解过程如下，每一步都有实测数据：

方案	解长	扩展结点	耗时	内存	是否达标
A* 配曼哈顿距离	—	—	—	OutOfMemory	否
A* 配模式数据库（5-5-5）	—	—	—	OutOfMemory（12 GB）	否
IDA* 配模式数据库（5-5-5，h=52）	70（最优）	10.24 亿	828.25 s	516 MB	否（超 1 分钟）
IDA* 配组合启发（h=56）	70（最优）	2.63 亿	369.35 s	516 MB	否（超 1 分钟）

IDA* 配组合启发,加权 w=1.2	72	155 万	2.62 s	516 MB	是
IDA* 配组合启发,加权 w=1.3	74	49.8 万	2.47 s	516 MB	是
IDA* 配组合启发,加权 w=1.5	84	1.87 万	2.36 s	516 MB	是

这组数据里有几件事要说清楚。

先说不加权的版本。IDA* 的启发函数可采纳，只要搜完就一定是最优解：配 5-5-5 模式数据库能求出 **70 步最优解**，代价是 828 秒。这一步的意义是把该实例从"A* 完全跑不动"推进到"能解出来"。

再换组合启发函数，初始状态的估计值从 52 提到 56，扩展结点少了 74%，耗时从 828 秒压到 369 秒，解仍然是最优的 70 步。这和 4.3.1 是同一个结论：瓶颈在启发函数的强度上。

不过 369 秒离 1 分钟还差 6 倍多，而 4.3.3 已经试过加大模式组，那条路走不通。

最后达标靠的是**加权 IDA***：令 $f = g + w \cdot h$（$w > 1$），让搜索更贪婪地朝目标方向走。

加权有一个代价要说清楚：**加权之后不再保证最优**。实测 $w = 1.2$ 时解长 72 步，比最优多 2 步，但耗时从 369 秒降到 **2.62 秒，快了 141 倍**。

两种结果我们都留着：

标准 IDA 配模式数据库，给出 70 步最优解，用时 369 秒；*

加权 IDA（w=1.2），给出 72 步解，用时 2.62 秒。*

要最优就接受 369 秒，要 1 分钟达标就用 $w = 1.2$ 的 72 步解，差别全在 w 上。

还有一步不能省：加权改变了剪枝策略，所以对每个加权结果我们都做了**独立回放验证**——从初始盘面重新执行解路径里的每一步，检查两件事：每一步移动是否合法（空格没越界）、最终盘面是否恰好等于目标盘面。$w = 1.2$、1.3、1.5 三个结果**全部通过**。

5. 解路径的可视化

5.1 设计目标与总体结构

第四阶段要求做"解路径的可视化"。我们用 **JDK 自带的 Swing** 实现（不引入任何第三方依赖），整体分成三层，各管一摊：

层	类	职责
数据层	`NPuzzleProblem.showSolution(path)`	框架的回调钩子，把解路径拆成"一串盘面"，每一帧一张
绘制层	`BoardPanel extends JPanel`	只管"把一块盘面画出来"，不关心现在是第几帧、有没有在播放
控制层	`PuzzleVisualizer extends JFrame`	只管"动画"：帧推进、播放暂停、速度、交互

这么分层的好处很实际：BoardPanel 不知道自己画的是第几帧，PuzzleVisualizer 不知道格子该怎么画。两边都因此变得简单，改一边也不会牵动另一边。

5.2 绘制层：BoardPanel 的关键逻辑

自定义绘制：重写 paintComponent(Graphics)，用 Graphics2D 打开抗锯齿，逐格画圆角矩形。

坐标自适应：先取面板宽高中**较小的那个**，减掉内边距，再除以阶数，得到当前帧的格子边长：

```
int usable = Math.min(getWidth(), getHeight()) - 2 * PADDING;
int cell = usable / n;                    // 每格边长，随窗口大小自动变化
int boardPx = cell * n;
int x0 = (getWidth() - boardPx) / 2;       // 居中
int y0 = (getHeight() - boardPx) / 2;
```

所以拖动窗口时棋盘会自己等比缩放并保持居中，一行 resize 监听都不用写。

三态配色：

格子	画法	含义
空格（0）	深色空槽	可以移动的位置
已归位的将牌	绿色渐变	位置与目标盘面一致，一眼就能看出"对了几块"
未归位的将牌	蓝色渐变	还需要移动

高亮刚移动的那一块：比较相邻两帧，找出"上一帧是空格、这一帧是将牌"的格子，给它加一圈琥珀色描边：

```
private static int movedTileIndex(PuzzleBoard prev, PuzzleBoard cur) {
    // 上一帧这里是空格、这一帧变成了将牌，就是刚滑过来的那一块
    if (prev.tileAt(r, c) == 0 && cur.tileAt(r, c) != 0) return r * n + c;
}
```

这样观众能看清"这一步到底动了哪一块"。

5.3 控制层：PuzzleVisualizer 的关键逻辑

动画驱动用 javax.swing.Timer：它按固定间隔在 **EDT（事件分发线程）** 上回调，推进动画天然线程安全，既不用自己开线程，也不用加锁。

Swing 的线程规矩：所有组件的创建和显示都得在 EDT 上做，所以对外入口用 SwingUtilities.invokeLater：

```
public static void showAndWait(...) {
    CountDownLatch closed = new CountDownLatch(1);
    SwingUtilities.invokeLater(() -> {
        PuzzleVisualizer window = new PuzzleVisualizer(...);
        window.addWindowListener(new WindowAdapter() {
```

```
            @Override      public      void      windowClosed(WindowEvent      e)
{ closed.countDown(); }
        });
        window.setVisible(true);
    });
    closed.await();     // 阻塞调用线程，直到用户关掉窗口
}
```

这里没有用 invokeAndWait，因为它只等"窗口显示完成"就返回，用户还没看呢。改用 CountDownLatch 把调用线程挂住，由窗口的 windowClosed 事件解锁，就实现了"**关掉窗口才继续**"。这也是批量演示能自动串联的关键。

交互设计：

操作	**方式**
播放 / 暂停	按钮或**空格键**
单步前进 / 后退	按钮或 **← / →**
回到初始 / 跳到目标	**Home / End**
调速	滑块（每帧 60 ms ~ 900 ms）
缩放棋盘	直接拖窗口，棋盘自动等比缩放

键盘绑定用的是 Swing 的 InputMap/ActionMap（WHEN_IN_FOCUSED_WINDOW），不受按钮焦点切换影响。

批量演示模式：第 6 个参数支持 3（单个）、2-6（区间）、all（全部），关掉当前窗口就自动弹下一个，全部播完控制台打一张汇总表。这是为阶段二（7 个 4 阶实例）和阶段三（需要用 IDA*）准备的。

与框架的两种集成方式：一是独立入口 PuzzleVisualizer.main(...)，参数与 SearchTester 前 4 位一致，后面可以追加启发函数、实例范围、--ida；二是挂到框架的钩子上，给 JVM 加 -Dnpuzzle.gui=true，NPuzzleProblem.showSolution 就会改为弹窗，否则打印控制台文字版。

5.4 演示效果

演示	**命令要点**	**效果**
阶段一（3 阶）	problems_stage1.txt … MANHATTAN all	3 个实例连播，秒开
阶段二（4 阶）	problems_stage2.txt … DISJOINT_PATTERN 1-6	6 个实例连播，绿色将牌一块块变多
阶段三（倒序实例）	problems_stage3.txt … DISJOINT_PATTERN --ida 加 -Dnpuzzle.weight=1.2	72 步长动画，从"几乎全蓝"到"全部归位"，演示效果最好

窗口顶部会实时显示实验数据，比如：

```
实例 1 / 1   4 阶   IDA* (w=1.2) + DISJOINT_PATTERN
解路径 72 步   耗时 2.950 s   生成 1554733 结点 / 扩展 529218 结点
```

我们实测确认窗口在三类实例上都能正常渲染（4 阶两位数编号对齐正确、颜色和高亮正常），并且做了**无图形环境下的优雅降级——**headless 时打印一行提示然后继续把批量实验跑完。

6. 总结及展望

6.1 完成情况对照

阶段	要求	结果
一	1 秒内解出 3 阶实例	最慢 0.156 s，余量 6 倍以上
二	4 阶与参考程序同级别时间	A* 的解长与扩展结点数与参考逐位一致；数据结构的 $O(1)$ 哈希优化让 IDA* 比参考快约 3 倍
三	1 分钟内解出倒序实例	2.62 s 达标（加权 IDA* 配模式数据库，72 步）；同时给出 70 步最优解，用时 369 秒
四	解路径可视化	Swing 窗口加批量演示，3 阶和 4 阶都验证通过

6.2 主要收获

这个项目里最实在的一条经验是：瓶颈要先测，不能猜。三轮优化每一轮都留了量化对比，也纠正了两次想当然的判断。一次是以为 $O(N)$ 的 remove 拖慢了程序，实测只占 4%~14%；一次是以为模式组越大越强，实测 6-6-3 反而更弱、还慢了 13.7 倍。凭直觉找瓶颈，命中率并不高。

第二条是启发函数的强度和成本要一起看。数据结构优化最多带来 2.4 倍，换个启发函数是 100 倍以上，量级差得很远；但启发函数变强也意味着单次估值更贵，我们那版未优化的线性冲突就让总耗时多了 28%，后来把每块将牌的目标行列预计算成小表才把这块补回来。

第三条是内存和时间的取舍绕不开。同一个倒序实例，A* 配模式数据库给到 12 GB 还是 OOM，换 IDA* 之后只占 516 MB。

最后一条是做完实验才真正相信的：光推理不够，结论得跑出来验证。加权 IDA* 的三个结果我们都做了回放验证（逐步检查移动合法性、检查终局是否等于目标），6 个 A* 实例和 9 个 IDA* 实例也都和老师的参考程序逐项对过。

6.3 不足与展望

不足	可以改进的方向

模式数据库的单调性没能严格证明，而 A* 用的是"不重新开放"策略，理论上存在次优风险（实验中未观察到）	补上单调性证明，或者在 A* 里实现"重新开放"
阶段三的 1 分钟达标依赖加权，放弃了最优性	换更强的可采纳启发函数（7-8 划分、多套模式数据库取 max、双向搜索），有望在保持最优的前提下达标
模式数据库的建表和查询还没做增量或压缩优化（目前固定 33 MB、每组约 0.7 秒）	距离表位压缩、按需分块加载
"不在位将牌数"在 4 阶上不可用	由模式数据库取代

7. 参考文献

1. Russell S, Norvig P. 人工智能：一种现代的方法（第 3 版）. 清华大学出版社.
2. Korf R E. Finding Optimal Solutions to Rubik's Cube Using Pattern Databases. AAAI, 1997.
3. Korf R E, Felner A. Disjoint Pattern Database Heuristics. Artificial Intelligence, 2002, 134(1-2): 9-22.
4. Culberson J C, Schaeffer J. Pattern Databases. Computational Intelligence, 1998, 14(3): 318-334.
5. Edelkamp S, Schrödl S. Heuristic Search: Theory and Applications. Morgan Kaufmann, 2011.
6. Sedgewick R, Wayne K. Algorithms, 4th Edition（algs4 工具类来源）.
7. 课程资料：项目一 N-Puzzle 问题求解要求文档、实验指导书、SearchFramework 2.1 编程框架.

备注：*本文第 1、2 章中关于可采纳性与单调性的定义及其证明、A* 最优性证明、IDA* 基本原理，以及 N-Puzzle 的规则描述、状态空间规模、逆序数可解性判据等内容，属于经典教材与文献中的既有结论，摘自文末所列参考文献；本项目的原创工作集中在第 3 章的工程实现、第 4 章的实验设计与数据分析、第 5 章的可视化设计。*